

AI Weekly Schedule Optimizer

CSC 480 Final Project Proposal

Project Overview

Our project is an AI-based weekly scheduling assistant that automatically creates a weekly plan for a user based on fixed events, flexible tasks, deadlines, priorities, and personal preferences. The goal is not simply to display a calendar, but to reason about how a user's week should be organized under a set of constraints.

A user will provide fixed commitments such as work shifts, classes, meetings, appointments, or planned events. They will also provide flexible tasks such as homework, studying, chores, exercise, errands, or project work. Each flexible task can have a duration, deadline, priority level, and optional time preferences. The system will then generate a weekly schedule that fits the flexible tasks around the fixed commitments while trying to satisfy as many constraints and preferences as possible.

Motivation

Many students and workers have schedules that include both fixed commitments and flexible responsibilities. Planning a week manually can become difficult when tasks have different deadlines, durations, priorities, and preferred time windows. A simple calendar application allows users to enter events, but it does not usually decide where flexible tasks should go or explain why a particular schedule is better than another.

Our project treats weekly planning as an AI search and constraint satisfaction problem. The system must find a valid assignment of tasks to time slots while avoiding conflicts and optimizing for user preferences.

The project is non-trivial because the scheduler is not just storing events; it is searching over many possible assignments while satisfying hard constraints and optimizing soft preferences.

AI Connection

This project connects most directly to search and constraint satisfaction. We can model the week as a collection of discrete time slots. Fixed events block certain slots, while flexible tasks must be assigned to available slots. Each task has a domain of possible start times, and the system must choose placements that satisfy constraints.

Examples of hard constraints include:

- Fixed events cannot be moved.
- Two events cannot overlap.
- A task must be completed before its deadline.
- A task must receive enough total scheduled time.
- A task can only be scheduled during the user's available hours.
- Breaks or travel buffers may be required between events.

Examples of soft constraints include:

- Higher-priority tasks should be scheduled earlier.
- The user may prefer not to work late at night.
- The user may prefer to group similar tasks together.
- The user may want to avoid long uninterrupted blocks.
- The user may prefer lighter schedules on certain days.

Because there may be many possible schedules, the system must search through possible assignments, detect conflicts, backtrack when necessary, and compare different valid schedules using a scoring function.

Proposed Approach

We plan to implement at least two scheduling approaches. First, we will implement a greedy baseline scheduler. This scheduler will place tasks in the first available valid time slots, likely ordered by deadline or priority. This gives us a simple comparison point.

Second, we will implement a constraint-based search scheduler using backtracking. The scheduler will attempt to assign each flexible task to a valid time block. If a task placement causes a conflict or prevents future tasks from being scheduled, the algorithm will backtrack and try a different assignment.

To improve the search, we may use heuristics such as:

- Schedule the most constrained tasks first.
- Schedule tasks with the earliest deadlines first.
- Schedule high-priority tasks before low-priority tasks.
- Prefer placements that leave larger continuous blocks open.
- Use forward checking to remove impossible future choices earlier.

We will compare the greedy approach to the backtracking and heuristic approach to show that the AI method is more effective than a simple calendar-filling strategy.

Inputs

The system will accept inputs such as:

- Fixed events: name, day, start time, end time.
- Flexible tasks: name, duration, deadline, priority.
- Availability: earliest and latest working times per day.
- Preferences: preferred days or times, break requirements, maximum daily workload.

Example fixed event: Work, Tuesday 2:00 PM-7:00 PM.

Example flexible task: Study for AI exam, 3 hours total, due Thursday, high priority, preferably before 8:00 PM.

Outputs

The output will be a complete weekly schedule showing where each flexible task was placed. The system should also report whether all tasks could be scheduled. If not, it should explain which constraints made the schedule impossible.

Possible output features include:

- A text-based weekly calendar.
- A simple visual calendar.
- A list of unscheduled tasks.
- A schedule quality score.
- Explanations for why tasks were placed at certain times.

Evaluation

We will evaluate the system using multiple test cases with different levels of difficulty. For example, we can test simple weeks with many open time slots, medium weeks with several fixed commitments, and difficult weeks where there are many tasks close to their deadlines.

We will compare the greedy baseline scheduler and the constraint-based scheduler using metrics such as:

- Number of tasks successfully scheduled.
- Number of hard constraint violations.
- Total preference score.
- Runtime.
- Ability to find a valid schedule when one exists.
- Quality of schedule compared to the baseline.

A successful project should show that the constraint-based scheduler handles more complex scheduling situations than the greedy baseline.

Stretch Goals

- Rescheduling when a new event is added.
- Recurring tasks.
- Task splitting across multiple days.
- Preference learning from user feedback.
- Multiple schedule options ranked by quality.
- A web interface or visual calendar.
- Natural-language input for tasks and events.

Expected Challenges

The main challenge will be designing a scheduling model that is complex enough to be interesting but still manageable to implement. We will need to decide how to represent time slots, tasks, constraints, and preferences. Another challenge will be balancing hard constraints, which must be satisfied, with soft

preferences, which should improve the schedule but may not always be possible.

We also expect that the search space may grow quickly as the number of tasks increases. Because of this, heuristics will be important for making the scheduler efficient enough to work on realistic weekly schedules.

Final Goal

By the end of the project, we aim to demonstrate an AI scheduler that can take a realistic set of weekly commitments and automatically produce a valid, optimized schedule. The project will show how search, constraint satisfaction, heuristics, and evaluation can be applied to a practical planning problem.